

A Comprehensive Guide to Writing Linux System Calls

Kernel Development Notes

August 27, 2026

Introduction

Writing kernel code is fundamentally different from writing user-space applications. The kernel operates in an isolated, highly privileged environment. A simple mistake here can panic (crash) the entire system. This guide covers the essential rules of writing kernel code and the end-to-end process of implementing a system call.

The Rules of Writing Kernel Code

Rule 1: No Standard Library

In user space, you rely on headers like `stdio.h`, `stdlib.h`, and `string.h`. In the kernel, **none of these exist**. Instead, you must use kernel equivalents provided by the kernel APIs:

- `printf()` → `printk()` (prints to the kernel log, viewable with `dmesg`)
- `malloc()` → `kmalloc(size, GFP_KERNEL)` (allocate kernel memory)
- `free()` → `kfree()` (free kernel memory)
- `sprintf()` → `snprintf()` (from `<linux/kernel.h>`)
- `strlen()` → `strlen()` (from `<linux/string.h>`)

Rule 2: The `SYSCALL_DEFINE` Macro

You never write a system call as a regular C function. You must use the `SYSCALL_DEFINE` macros. The number appended to the macro indicates the number of arguments the syscall takes.

Notice the unique syntax: the type and name of each argument are separated by commas.

```

1 // 0 arguments
2 SYSCALL_DEFINE0(my_syscall) { ... }
3
4 // 1 argument
5 SYSCALL_DEFINE1(my_syscall, int, arg1) { ... }
6
7 // 2 arguments
8 SYSCALL_DEFINE2(my_syscall, int, arg1, char __user *, buf) { ... }
9
10 // 4 arguments
11 SYSCALL_DEFINE4(proc_info, pid_t, pid, unsigned int, flags,
12                  char __user *, buffer, size_t, size) { ... }

```

Listing 1: Using SYSCALL_DEFINE macros

Rule 3: User Space Memory is Untouchable

This is the most critical security and stability rule. The kernel cannot blindly trust or directly dereference user-space pointers. You must copy data safely between user space and kernel space using specific functions:

- `copy_to_user(user_ptr, kernel_ptr, size)` – Copy FROM kernel TO user.
- `copy_from_user(kernel_ptr, user_ptr, size)` – Copy FROM user TO kernel.

These functions return 0 on success, and non-zero (the number of bytes that failed to copy) on failure. Always return `-EFAULT` if they fail.

```

1 char kernel_buf[256];
2 snprintf(kernel_buf, sizeof(kernel_buf), "hello from kernel");
3
4 if (copy_to_user(buffer, kernel_buf, strlen(kernel_buf) + 1))
5     return -EFAULT;

```

Listing 2: Safely copying data to user space

Rule 4: Finding a Process by PID

The kernel tracks all processes using a structure called `task_struct`. To find a specific process using its PID:

```

1 struct task_struct *task;
2 task = pid_task(find_vpid(pid), PIDTYPE_PID);
3 if (!task)
4     return -ESRCH; // Error: No such process

```

Listing 3: Locating a `task_struct` by PID

Once you have the `task_struct`, you can access process metadata:

- `task->pid` – The Process ID
- `task->parent->pid` – The Parent's PID
- `task->__state` – Current state (running, sleeping, etc.)
- `task->prio` – Priority

- `task->policy` – Scheduling policy (FIFO, RR, Normal)
- `task->nvcsw` – Voluntary context switches
- `task->nivcsw` – Involuntary context switches

Rule 5: Iterating All Processes

To loop through every active process on the system, use the `for_each_process` macro:

```
1 struct task_struct *task;
2 for_each_process(task) {
3     printk("PID: %d\n", task->pid);
4 }
```

Listing 4: Iterating through all processes

Rule 6: Kernel Linked Lists

The Linux kernel has its own highly efficient, intrusive doubly-linked list implementation defined in `<linux/list.h>`. You embed a `list_head` struct inside your custom node structure.

```
1 // Define the list head (global, static)
2 static LIST_HEAD(monitored_list);
3
4 // Your node structure
5 struct pid_node {
6     pid_t pid;
7     unsigned long ninvctx;
8     unsigned long nvctx;
9     struct list_head next_prev_list;
10 };
11
12 // Add node to the tail of the list
13 list_add_tail(&node->next_prev_list, &monitored_list);
14
15 // Iterate through the list
16 struct pid_node *entry;
17 list_for_each_entry(entry, &monitored_list, next_prev_list) {
18     printk("PID: %d\n", entry->pid);
19 }
20
21 // Delete from list and free memory
22 list_del(&node->next_prev_list);
23 kfree(node);
```

Listing 5: Using kernel linked lists

Rule 7: Error Codes

Syscalls return a `long` integer. Return 0 for success, and standard **negative** error codes for failures (defined in `<linux/errno.h>`):

- `-EINVAL` – Invalid argument

- -ESRCH – No such process
- -EFAULT – Bad user space address
- -ENOSPC – Not enough space
- -ENOMEM – Out of memory
- -EPERM – Operation not permitted

Rule 8: Essential Headers

You will frequently need the following headers when writing kernel code:

```

1 #include <linux/kernel.h>      // printk, snprintf
2 #include <linux/syscalls.h>    // SYSCALL_DEFINE
3 #include <linux/sched.h>       // task_struct, for_each_process
4 #include <linux/pid.h>         // find_vpid, pid_task
5 #include <linux/uaccess.h>     // copy_to_user, copy_from_user
6 #include <linux/list.h>        // linked list macros
7 #include <linux/slab.h>        // kmalloc, kfree

```

Quick Summary Table

Concept	User Space	Kernel Space
Printing	printf	printk
Memory Allocation	malloc / free	kmalloc / kfree
Pointer Access	Direct pointer manipulation	copy_to_user / copy_from_user
Error Handling	Return -1, set errno	Return negative codes (e.g., -EINVAL)
Standard Library	#include <stdio.h>	#include <linux/kernel.h>

Table 1: User Space vs. Kernel Space Equivalents

Registering and Calling Your Syscall

Writing the function is only half the battle. The kernel must be made aware of your new syscall so user-space applications can invoke it.

Step 1: Adding to the Syscall Table

The kernel assigns a unique integer to every system call. You must add your syscall to the syscall table. For x86_64 architectures, edit `arch/x86/entry/syscalls/syscall_64.tbl`:

```
1 # format: <number> <abi> <name> <entry point>
2 # Scroll to the bottom of the table and add:
3 451      common  my_syscall      sys_my_syscall
4 452      common  proc_info       sys_proc_info
```

Listing 6: Modifying syscall_64.tbl

Note: "451" is just an example. Use the next available number.

Step 2: Updating the Kernel Header

You must declare the function prototype so the rest of the kernel knows about it. Edit `include/linux/syscalls.h` and add it to the bottom (just before the final `#endif`):

```
1 asmlinkage long sys_my_syscall(int arg1, char __user *buf);
2 asmlinkage long sys_proc_info(pid_t pid, unsigned int flags,
3                               char __user *buffer, size_t size);
```

Listing 7: Modifying syscalls.h

The `asmlinkage` keyword tells the compiler to look on the CPU stack for the function arguments, rather than in registers.

Step 3: Compiling the Code (Makefile)

If you wrote your syscalls in a new file (e.g., `kernel/my_syscalls.c`), you must tell the kernel build system to compile it. Edit `kernel/Makefile`:

```
1 # Find the obj-y line and append your object file:
2 obj-y += my_syscalls.o
```

Listing 8: Modifying kernel/Makefile

After this, you must recompile the kernel (e.g., `make -j$(nproc)`), install the modules, and update your bootloader (GRUB).

Step 4: Calling the Syscall from User Space

Standard C libraries (like `glibc`) do not have wrapper functions for newly created custom syscalls. Therefore, you cannot simply call `my_syscall(123, buf);`. You must use the `syscall()` wrapper function provided by `<unistd.h>`.

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/syscall.h>
4
5 #define SYS_my_syscall 451 // The number from syscall_64.tbl
```

```
6
7 int main() {
8     char buffer[256];
9     long res = syscall(SYS_my_syscall, 123, buffer);
10
11     if (res < 0) {
12         perror("Syscall failed");
13         return -1;
14     }
15
16     printf("Kernel said: %s\n", buffer);
17     return 0;
18 }
```

Listing 9: User space invocation